جزوه دوره مهندسی داده و مهندسی هوش مصنوعی

Data Engineering & AI Engineering

یادداشت‌های آموزشی، مفاهیم فنی و تمرین‌های دوره



جلسه نهم: آموزش Pandas؛ از SQL تا دیتافریم

مدرس :سحر منصوری‌فر

تاریخ :۱۳ آبان ۱۴۰۵

تاریخ به روز رسانی : ۲۰۲۶۱۱۰۵

درباره این جزوه

این جزوه با هدف ثبت، سازمان‌دهی و بازنویسی آموزشی مطالب دوره رایگان «مهندسی داده و مهندسی هوش مصنوعی» تهیه شده است.

محتوای آن بر اساس توضیحات مدرس، مثال‌های مطرح‌شده در کلاس و پرسش‌وپاسخ‌های آموزشی تدوین می‌شود. تلاش شده است مطالب گفتاری کلاس به متنی منظم، قابل فهم و مناسب مطالعه مستقل تبدیل شوند، بدون اینکه مفهوم اصلی صحبت‌های مدرس تغییر کند.

هدف این جزوه صرفاً خلاصه‌کردن جلسات نیست؛ بلکه ایجاد یک منبع آموزشی است که دانشجو بتواند در طول دوره و حتی چند ماه پس از پایان آن، برای یادگیری دوباره مفاهیم، مرور نکات فنی و آماده‌شدن برای فعالیت‌های عملی به آن مراجعه کند.

رویکرد آموزشی و شیوه تدوین

در تدوین مطالب، ابتدا مفاهیم اصلی هر جلسه با زبانی ساده و دقیق توضیح داده می‌شوند. سپس مثال‌های مدرس، کاربردهای عملی و ارتباط موضوعات با مسائل واقعی مهندسی داده و هوش مصنوعی بررسی می‌شوند.

برای حفظ دقت علمی و تمایز میان محتوای اصلی کلاس و توضیحات تکمیلی، از ساختار زیر استفاده شده است:

- **مطالب اصلی جلسه :**بازنویسی منظم توضیحات مدرس با حفظ مفهوم و مثال‌های اصلی.
- **تکمیل فنی :**توضیحات دقیق‌تر، اصلاح برداشت‌های احتمالی و نکات تکمیلی بر اساس منابع فنی معتبر.
- **مثال اضافی :**مثال‌ها و سناریوهایی که برای درک بهتر مفاهیم به جزوه افزوده شده‌اند.
- **نکات کاربردی :**ملاحظات اجرایی، محدودیت‌ها و تجربه‌های مرتبط با استفاده عملی از مفاهیم.
- **جمع‌بندی و سوال‌های مرور :**مرور مهم‌ترین مطالب و ارزیابی میزان یادگیری در پایان هر جلسه.

اصطلاحات تخصصی مانندData Pipeline ، Data Warehouse، ETL، Orchestrationو سایر مفاهیم فنی با نام اصلی انگلیسی حفظ شده‌اند تا مطالعه مستندات رسمی و منابع تخصصی برای دانشجویان آسان‌تر باشد.

محدوده و توسعه جزوه

این مجموعه به‌صورت تدریجی و هم‌زمان با انتشار جلسات جدید تکمیل می‌شود و دو مسیر اصلی دوره، یعنی Data Engineering وAI Engineering ، را پوشش می‌دهد.

نسخه‌های به‌روزشده جزوه‌ها، منابع تکمیلی و یادداشت‌های هر جلسه از طریق مخزن عمومی GitHub در دسترس خواهند بود.

مخزن جزوه‌ها:

https://github.com/LaneZero/data-ai-engineering-course

وب‌سایت اصلی دوره:

https://mostafaghadimi.github.io/data-course/

یادداشت درباره اعتبار محتوا

این جزوه یک مجموعه آموزشی مستقل و غیررسمی است و وابستگی رسمی به مدرس یا برگزارکنندگان دوره ندارد.

حقوق محتوای اصلی دوره متعلق به صاحبان آن است. توضیحات تکمیلی و مثال‌های افزوده‌شده نیز لزوماً بیانگر دیدگاه یا تأیید مدرس نیستند.

با وجود تلاش برای حفظ دقت فنی، توصیه می‌شود هنگام اجرای پروژه‌های واقعی، اطلاعات حساس یا تصمیم‌گیری‌های فنی مهم، مستندات رسمی ابزارها و منابع اصلی نیز بررسی شوند.

هدف از انتشار این مجموعه، **کمک به یادگیری عمیق‌تر، مرور منظم مفاهیم و اشتراک دانش با علاقه‌مندان حوزه داده و هوش مصنوعی** است.

فهرست مطالب

۱. هدف جلسه و نقشه راه .Error! Bookmark not defined

۲. مرور تمرین: تعداد سفارش به تفکیک وضعیت .Error! Bookmark not defined

۳. تفکر مجموعه‌ای در .SQL Error! Bookmark not defined

۴. Window Function: محاسبه بدون از بین بردن ردیف‌ها .Error! Bookmark not defined

۴.۱ شماره سفارش، سفارش قبلی و جمع تجمعی .Error! Bookmark not defined

۴.۲ مجموع و درصد هر سفارش از خرید مشتری .Error! Bookmark not defined

۴.۳ پیدا کردن اولین سفارش هر مشتری .Error! Bookmark not defined

۵. شش خطای رایج و خطرناک در .SQL Error! Bookmark not defined

۵.۱ خطای اول: Fan-out پس از .JOIN Error! Bookmark not defined

۵.۲ خطای دوم: تبدیل ناخواسته LEFT JOIN به فیلتر محدودکننده .Error! Bookmark not defined

۵.۳ خطای سوم: NULL و منطق سه‌ارزشی .Error! Bookmark not defined

۵.۴ خطای چهارم: NOT IN و حضور NULL در زیرپرس‌وجو .Error! Bookmark not defined

۵.۵ خطای پنجم: بازه زمانی و دقت .timestamp Error! Bookmark not defined

۵.۶ خطای ششم: UPDATE بدون شرط و بررسی اثر تغییر .Error! Bookmark not defined

۶. تله‌های عددی و تقسیم بر صفر .Error! Bookmark not defined

۷. ساخت کوئری پارامتری و جلوگیری از خطاهای امنیتی .Error! Bookmark not defined

۸. الگوهای تحلیلی که مدرس برای تمرین معرفی کرد .Error! Bookmark not defined

۸.۱ رتبه‌بندی: ROW_NUMBER, RANK و .DENSE_RANK Error! Bookmark not defined

۸.۲ حذف تکراری‌ها .Error! Bookmark not defined

۸.۳ رشد ماهانه و جمع تجمعی .Error! Bookmark not defined

۸.۴ قیف مشتری، Point-in-Time و الگوهای پیشرفته .Error! Bookmark not defined

۹. پیوند SQL با مهندسی داده: Grain, Fact و .Dimension Error! Bookmark not defined

۱۰. چک‌لیست پیش از تأیید .Query Error! Bookmark not defined

۱۱. تجربه مدرس و توصیه‌های پایانی .Error! Bookmark not defined

۱۲. جمع‌بندی .Error! Bookmark not defined

۱۳. سوال‌های مرور .Error! Bookmark not defined

۱۴. پاسخ‌نامه کوتاه .Error! Bookmark not defined

۱۵. واژه‌نامه .Error! Bookmark not defined

. هدف جلسه و مسیر یادگیری

این جلسه یک کارگاه مقدماتی Pandas است. مدرس کار را با Google Colab و واردکردن کتابخانه آغاز می‌کند، داده‌های نمونه Titanic را می‌خواند و سپس انتخاب سطر و ستون، ساخت ستون، پاک‌سازی، فیلتر، ادغام جدول‌ها و تجمیع را به‌صورت عملی نمایش می‌دهد. پیوند اصلی با دو جلسه SQL قبلی این است که بفهمیم پرسش‌هایی مانند «کدام مسافران؟»، «میانگین سن هر گروه؟» و «چگونه دو جدول را ترکیب کنیم؟» در DataFrame چگونه بیان می‌شوند.

پس از مطالعه باید بتوانید DataFrame بسازید، اطلاعات اولیه آن را بررسی کنید، فرق `loc` و `iloc` را توضیح دهید، فیلترهای چندشرطی بنویسید، تفاوت `concat` و `merge` را تشخیص دهید و پیش از حذف داده‌های ناقص یا تکراری اثر تصمیم خود را بسنجید.

۲. Pandas چیست و چه نسبتی با SQL دارد؟

Pandas کتابخانه‌ای در Python برای خواندن، تمیزکردن، تبدیل و تحلیل داده‌های جدولی است. دو ساختار رایج آن `Series` برای داده یک‌بعدی و `DataFrame` برای جدول دوبعدی هستند. در SQL غالباً پرسش را به موتور دیتابیس می‌سپاریم؛ در Pandas روی شیء DataFrame در محیط Python عملیات انجام می‌دهیم. مدرس Pandas را به‌عنوان راهی برای اجرای کارهای مشابه Excel و SQL معرفی می‌کند.

نیاز تحلیلی	SQL	Pandas
انتخاب ستون	`SELECT name, age`	`df[['Name', 'Age']]`
شرط روی ردیف	`WHERE age > 50`	`df.loc[df['Age'] > 50]`
گروه‌بندی	`GROUP BY sex`	`df.groupby('Sex')`
اتصال بر اساس کلید	`JOIN ... ON id`	`pd.merge(..., on='Id')`
افزودن ردیف‌های جداول	`UNION ALL`	`pd.concat(..., axis=0)`

تکمیل فنی: Pandas جایگزین عمومی پایگاه داده نیست؛ اجرای آن معمولاً به حافظه ماشین و ظرفیت پردازش محلی محدود می‌شود. برای داده بسیار بزرگ، خواندن انتخابی از پایگاه داده، پردازش تکه‌ای یا ابزارهای توزیع‌شده ممکن است مناسب‌تر باشند. همچنین `UNION` در SQL رکورد تکراری را حذف می‌کند، اما `concat` به‌خودی‌خود چنین کاری نمی‌کند؛ مقایسه درست‌تر با `UNION ALL` است.

۳. محیط کار: Google Colab و Notebook

در کلاس، کد در Google Colab اجرا شد. Notebook از سلول‌های کد و متن Markdown تشکیل می‌شود؛ هر سلول را می‌توان مستقلاً اجرا کرد و خروجی‌اش را دید. این ویژگی برای آزمودن مرحله‌ای تبدیل داده مفید است، اما ترتیب اجرا مهم است: سلولی که قبلاً اجرا شده ممکن است متغیری را در حافظه نگه داشته باشد که در اجرای مجدد از ابتدا وجود ندارد.

```
# Import the library once per notebook session.
import pandas as pd
print(pd.__version__)
```

برای محیط محلی در صورت نصب نبودن بسته:

```
python -m pip install pandas
```

> **تکمیل فنی:** در Colab هم نمی‌توان نصب بودن هر کتابخانه یا پابرجاماندن داده‌ها را تضمین کرد. محیط اجرایی موقت است و پس از Reset ممکن است فایل‌های آپلودشده حذف شوند؛ برای بازتولیدپذیری، ورودی‌ها و نسخه وابستگی‌ها را ثبت کنید.

۴. ساخت مجموعه‌داده نمونه و خواندن فایل

مدرس فایل CSV نمونه Titanic را در محیط Colab آپلود کرده و با تابع `read_csv` آن را به DataFrame تبدیل می‌کند. فایل می‌تواند به‌جای CSV در قالب Excel یا JSON باشد که هرکدام تابع خواندن مناسب خود را دارند.

```
# Source file must exist in the notebook's working directory.
df = pd.read_csv('titanic.csv')
print(df.head())
```

اگر فایل جلسه در دسترس نیست، نمونه زیر برای اجرای باقی تمرین‌ها کافی است. اعداد و رکوردها **ساختگی** هستند:

```
import pandas as pd

# Synthetic Titanic-like data for reproducible exercises.
df = pd.DataFrame({
    'PassengerId': [1, 2, 3, 4, 5, 6],
    'Name': ['Ali', 'Sara', 'Reza', 'Neda', 'Mina', 'Omid'],
    'Sex': ['male', 'female', 'male', 'female', 'female', 'male'],
    'Age': [22, 38, None, 55, 19, 55],
    'Pclass': [3, 1, 2, 1, 3, 2],
    'Survived': [0, 1, 0, 1, 1, 0],
})
```

۴.۱ ساخت DataFrame از دیکشنری

در پاسخ به پرسش یکی از دانشجویان، مدرس نشان می‌دهد که داده لازم نیست حتماً از فایل خوانده شود؛ می‌توان دیکشنری‌ای ساخت که کلیدهایش نام ستون و مقدارهایش فهرست داده باشند، سپس از `pd.DataFrame` استفاده کرد.

```
people = {'Name': ['Sara', 'Ali'], 'Age': [27, 32]}
people_df = pd.DataFrame(people)
```

> **تکمیل فنی:** طول فهرست‌های ستون‌ها باید یکسان باشد. برای داده‌های واقعی بهتر است نوع داده، Encoding، جداکننده CSV و شیوه مدیریت مقادیر گمشده را هنگام خواندن بررسی کنید.

۵. شناخت سریع ساختار DataFrame

مدرس پیش از تحلیل، چند تابع را برای شناخت اولیه مجموعه‌داده اجرا می‌کند: `head` و `tail` برای دیدن ابتدای و انتهای جدول؛ `sample` برای نمونه تصادفی؛ `info` برای آگاهی از تعداد مقدارهای غیرخالی و نوع ستون‌ها؛ `describe` برای آمار توصیفی؛ و `shape` و `columns` برای ابعاد و نام ستون‌ها.

```
print(df.head())        # First 5 rows
print(df.tail(3))       # Last 3 rows
print(df.sample(2, random_state=42))
df.info()               # Prints types and non-null counts
print(df.describe())    # Numeric summary by default
print(df.shape)         # (row_count, column_count)
print(df.columns.tolist())
print(df['Age'].describe())
```

دستور	نتیجه اصلی	نکته
`head(n)`	`n` ردیف اول	پیش‌فرض ۵ است
`tail(n)`	`n` ردیف آخر	نمایی سریع از انتها
`sample(n)`	نمونه تصادفی	با `random_state` تکرارپذیر
`info()`	نوع و تعداد مقدار غیرخالی	معمولاً خروجی را چاپ می‌کند
`describe()`	شاخص‌های توصیفی	به‌طور پیش‌فرض ستون‌های عددی
`shape`	تعداد ردیف و ستون	ویژگی است، تابع نیست
`columns`	برچسب ستون‌ها	برای بررسی غلط‌های نام‌گذاری

> **تکمیل فنی:** `info()` و `describe()` معادل دقیق هم نیستند. `shape` تعداد ستون‌ها را مشخص می‌کند، نه لزوماً تعداد «ویژگی‌های قابل استفاده» برای مدل. نوع `object` نیز الزاماً به معنی رشته تمیز و یک‌دست نیست.

۶. انتخاب ستون‌ها و سطرها

برای یک ستون می‌توان از `df['Age']` استفاده کرد که معمولاً یک Series می‌دهد. انتخاب چند ستون با دو براکت صورت می‌گیرد و نتیجه DataFrame است:

```
ages = df['Age']
subset = df[['Name', 'Age', 'Survived']]
print(subset.head())
```

۶.۱ تفاوت `iloc` و `loc`

`iloc` انتخاب مبتنی بر **موقعیت عددی** است؛ `loc` انتخاب مبتنی بر **برچسب Index و نام ستون** است. مدرس بازه‌هایی از سطرها و ستون‌ها را با این دو روش آزمایش کرد.

```
# Positional: rows 0..4, columns 0..2 (end excluded).
first_block = df.iloc[0:5, 0:3]

# Label-based: labels 1..3 are included when present.
selected = df.loc[1:3, ['Name', 'Age']]

# Single row by position; selected fields by names.
one_row = df.loc[2, ['Name', 'Pclass']]
```

> **تکمیل فنی:** در `iloc[0:5]` عدد ۵ داخل بازه نیست، اما در `loc[1:3]` وقتی برچسب‌های ۱ تا ۳ وجود داشته باشند، برچسب ۳ هم انتخاب می‌شود. اگر Index پس از فیلتر/حذف ردیف تغییر کرده باشد، عددِ داخل `loc` الزاماً همان موقعیت سطر نیست. این تفاوت از خطاهای رایج کلاس‌های مقدماتی است.

۷. افزودن، محاسبه و حذف ستون

مدرس دو حالت را نشان می‌دهد: تخصیص یک مقدار ثابت به همه ردیف‌ها و ساخت یک ستون جدید با محاسبه از ستون موجود، مثلاً چندبرابر کردن سن برای نمایش مکانیزم عملیات برداری.

```
work = df.copy()  # Preserve the original DataFrame.
work['NewColumn'] = 1
work['AgeTimes100'] = work['Age'] * 100
print(work[['Age', 'AgeTimes100']].head())
```

```
# Drop a named column and keep the returned new DataFrame.
work = work.drop(columns=['NewColumn'])
```

تکمیل فنی: ضرب سن در ۱۰۰ در کلاس صرفاً نمونه ساخت ستون است، نه یک تبدیل تحلیلی معنادار. استفاده از `df.copy()` در مثال بالا به جلوگیری از تغییر ناخواسته دیتافریم اصلی کمک می‌کند. `drop` بدون انتساب مجدد یا `inplace=True` معمولاً شیء اولیه را تغییر نمی‌دهد.

۸. فیلترکردن داده و شرط‌های چندگانه

در بخش شرط‌گذاری، مدرس مسافران زن کلاس اول و افراد خارج از یک بازه سنی را مثال می‌زند. در Pandas شرط‌ها ماسک بولی تولید می‌کنند و برای ترکیبشان از `&` (و)، `|` (یا) و `~` (نقیض) استفاده می‌شود.

```
# Female passengers in first class.
female_first = df.loc[
    (df['Sex'] == 'female') & (df['Pclass'] == 1)
]

# Younger than 20 OR older than 50.
outside_age_range = df.loc[
    (df['Age'] < 20) | (df['Age'] > 50)
]

print(female_first[['Name', 'Age']])
print(outside_age_range[['Name', 'Age']])
```

تکمیل فنی: برای Seriesهای Pandas از `and` و `or` معمول Python استفاده نکنید؛ این عملگرها برای یک مقدار True/False واحد طراحی شده‌اند و معمولاً خطای ابهام ارزش بولی می‌دهند. دور هر شرط پرانتز بگذارید. مقایسه با مقدار گمشده اغلب False یا مقدار بولی نامشخص تولید می‌کند؛ نتیجه را آگاهانه بررسی کنید.

۸.۱ جست‌وجوی متن و تبدیل نوع داده

مدرس به جست‌وجوی زیررشته با `str.contains` و تبدیل تاریخ‌ها اشاره می‌کند. نمونه تکمیلی:

```
# Match text safely, including missing values.
name_hits = df.loc[df['Name'].str.contains('sa', case=False, na=False)]

# Convert a date column when such a column exists.
orders = pd.DataFrame({'CreatedAt': ['2026-11-04', 'invalid']})
orders['CreatedAt'] = pd.to_datetime(orders['CreatedAt'], errors='coerce')
```

مثال اضافی: اگر تاریخ یک سفارش به‌صورت رشته ذخیره شود، مقایسه رشته‌ای همیشه جایگزین مقایسه زمانی نیست. با `pd.to_datetime` ابتدا نوع داده را معتبر کنید و موارد نامعتبر (`NaT`) را جداگانه بررسی کنید.

۹. ترکیب DataFrameها: `concat` و `merge`

مدرس دو نیاز متفاوت را کنار هم می‌گذارد: چسباندن دو جدول مشابه در ادامه هم (`concat`) و پیدا کردن ردیف‌های مرتبط از دو جدول بر اساس کلید (`merge`).

```
left = pd.DataFrame({'Id': [1, 2], 'Name': ['Ali', 'Sara']})
right = pd.DataFrame({'Id': [2, 3], 'Age': [30, 25]})

# Relational join on a shared key.
joined = pd.merge(left, right, on='Id', how='left', validate='one_to_one')

# Append same-schema rows vertically.
new_people = pd.DataFrame({'Id': [4], 'Name': ['Neda']})
stacked = pd.concat([left, new_people], axis=0, ignore_index=True)

print(joined)
print(stacked)
```

نوع اتصال	کدام کلیدها باقی می‌مانند؟	متناظر SQL
`how='inner'`	مشترک میان دو جدول	اتصال داخلی (`INNER JOIN`)
`how='left'`	همه کلیدهای جدول چپ	اتصال چپ (`LEFT JOIN`)
`how='right'`	همه کلیدهای جدول راست	اتصال راست (`RIGHT JOIN`)
`how='outer'`	اجتماع کلیدها	اتصال کامل (`FULL OUTER JOIN`)

تکمیل فنی: جدول دارای کلید تکراری الزاماً نامعتبر نیست، ولی در `merge` می‌تواند تعداد ردیف‌های خروجی را چندبرابر کند (Fan-out). پارامتر `validate` برای کنترل نوع رابطه مفید است. برخلاف رفتار معمول SQL, Pandas هنگام `merge` ممکن است کلیدهای Null دو طرف را با هم تطبیق دهد؛ پیش از اتصال، کلیدهای خالی را بررسی کنید.

۹.۱ `concat` افقی و عمودی

با `axis=0` جدول‌ها زیر هم قرار می‌گیرند و ستون‌ها براساس نام هم‌تراز می‌شوند؛ با `axis=1` جدول‌ها کنار هم قرار می‌گیرند و **بر اساس Index** هم‌تراز می‌شوند.

```
# Side-by-side concatenation by index, not by business key.
side_by_side = pd.concat(
    [left.reset_index(drop=True), right[['Age']].reset_index(drop=True)],
    axis=1,
)
```

۱۰. گروه‌بندی و محاسبات آماری

در کلاس میانگین سن به تفکیک جنسیت با `groupby` محاسبه شد و به توابع `min`، `max`، `sum`، `count` و ترکیب چند تابع با `agg` اشاره شد. در SQL همین مسئله با `GROUP BY` و `AVG` بیان می‌شود.

```
# Average passenger age per sex.
avg_age = df.groupby('Sex')['Age'].mean()

# Multiple metrics for a single numeric field.
summary = df.groupby('Sex')['Age'].agg(['count', 'mean', 'min', 'max'])
print(summary)
```

```
-- SQL equivalent on a passenger table.
SELECT Sex,
       COUNT(Age) AS age_count,
       AVG(Age) AS mean_age,
       MIN(Age) AS min_age,
       MAX(Age) AS max_age
FROM passengers
GROUP BY Sex;
```

تکمیل فنی: `count` در Pandas فقط مقادیر غیرخالی ستون را می‌شمارد؛ برای شمارش تمام ردیف‌های هر گروه از `size()` استفاده کنید. `mean()` نیز معمولاً مقدارهای گمشده را نادیده می‌گیرد. «میانگین سن زن و مرد» تا زمانی قابل تفسیر است که تعداد مقدارهای معتبر هر گروه هم معلوم باشد.

۱۱. پاک‌سازی داده: خالی‌ها و تکراری‌ها

مدرس `isna`، `dropna`، `fillna` و حذف داده‌های تکراری را معرفی می‌کند. اصل مهم این است که قبل از حذف داده، بدانیم چه چیزی حذف می‌شود و چرا.

```
# Count missing values per field.
print(df.isna().sum())

# Remove only rows without age, keeping other rows.
known_age = df.dropna(subset=['Age'])

# Fill missing ages for a separate analysis copy.
filled = df.copy()
filled['Age'] = filled['Age'].fillna(filled['Age'].median())

# Detect duplicate business identifiers and drop cautiously.
print(df.duplicated(subset=['PassengerId']).sum())
unique_passengers = df.drop_duplicates(subset=['PassengerId'], keep='first')
```

تکمیل فنی: `dropna()` بدون `subset` ممکن است هر سطری را که حتی یک مقدار خالی دارد حذف کند. `fillna` و `drop_duplicates` هم انتخاب‌های تحلیلی هستند، نه اقداماتی که همیشه درست باشند. تکراری بودن نام مسافر به‌تنهایی دلیل کافی برای حذف رکورد نیست؛ کلید کسب‌وکاری مناسب لازم است.

۱۱.۱ مرتب‌سازی و زنجیره عملیات

در بخش پایانی، مدرس نشان می‌دهد که می‌توان DataFrame را فیلتر و سپس ستون‌های موردنیاز را انتخاب و براساس سن مرتب کرد. `query` نیز برای بیان برخی شرط‌ها با عبارتی متنی معرفی شد.

```
# Readable chained analysis; no mutation of df.
result = (
    df.loc[(df['Sex'] == 'female') & (df['Age'] > 50), ['Name', 'Age']]
      .sort_values('Age', ascending=False)
)
print(result)

# Alternative expression with query on simple column names.
result_query = df.query("Sex == 'female' and Age > 50")
```

تکمیل فنی: در `DataFrame.query` نباید ورودی متنیِ کنترل‌نشده کاربران را بدون بررسی وارد عبارت کنید؛ برای برنامه‌های واقعی ابتدا ورودی را اعتبارسنجی کنید و ترجیحاً منطق فیلتر را با شرط‌های مشخص DataFrame بسازید.

۱۲. Pivot Table: اشاره پایانی کلاس

در انتهای جلسه مدرس به ساخت Pivot Table برای خلاصه‌سازی داده اشاره می‌کند، اما نمایش زنده آن در Transcript با خطا/قطع ادامه پیدا کرده و نمونه نهایی کامل ثبت نشده است. برای درک مفهوم، نمونه مستقل زیر تعداد مسافران در هر کلاس و جنسیت را نشان می‌دهد.

```
# Additional example; not a verified transcript demo.
pivot = pd.pivot_table(
    df,
    index='Pclass',
    columns='Sex',
    values='PassengerId',
    aggfunc='count',
    fill_value=0,
)
print(pivot)
```

مثال اضافی: Pivot در اینجا نقش یک جدول خلاصه دوبعدی را دارد. اگر یک گروه اصلاً رکورد نداشته باشد، `fill_value=0` صرفاً مقدار نمایشی خانه بدون مقدار را صفر می‌کند؛ لزوماً به معنی وجود داده واقعی نیست.

۱۳. سناریوی جامع: تحلیل قابل اجرا روی داده نمونه

هدف: تعداد مسافر، میانگین سن و نرخ بقا را به تفکیک کلاس محاسبه کنیم. در این نمونه فرضی، ستون `Survived` صفر و یک است؛ بنابراین میانگین آن نسبت افراد با مقدار یک را نشان می‌دهد.

```
# All inputs are from the synthetic DataFrame earlier in this note.
report = (
    df.groupby('Pclass', as_index=False)
      .agg(
          passengers=('PassengerId', 'size'),
          known_ages=('Age', 'count'),
          average_age=('Age', 'mean'),
          survival_rate=('Survived', 'mean'),
      )
      .sort_values('Pclass')
)
report['survival_percent'] = (report['survival_rate'] * 100).round(1)
print(report)
```

تکمیل فنی: این گزارش فقط برای آموزش GroupBy و Named Aggregation است. نرخ بقا در دیتاست ساختگی شش‌ردیفی هیچ ارزش آماری درباره حادثه واقعی ندارد؛ نتایج آن را تعمیم ندهید.

۱۴. مشکلات رایج و راه عیب‌یابی

نشانه	علت محتمل	اقدام پیشنهادی
`FileNotFoundError`	فایل در مسیر فعلی نیست	مسیر و فایل‌های Colab را بررسی کنید
`KeyError: 'Age'`	غلط تایپی یا تفاوت حروف نام ستون	`df.columns.tolist()`
خطای `ambiguous truth value`	استفاده از `and` به‌جای `&`	شرط‌های پرانتزبندی‌شده
خروجی `merge` بیش‌ازانتظار	کلید تکراری یا اتصال چندبه‌چند	`validate` و بررسی شمارش کلید
`NaN` پس از `concat`	ستون‌Index/ ناهماهنگ	ستون‌ها و Index را مقایسه کنید
نتیجه غیرمنتظره `loc`	تفاوت Label با Position	`df.index` را بررسی کنید
کاهش شدید ردیف‌ها	`dropna` یا `inner join` نامناسب	شمارش قبل و بعد هر مرحله

۱۵. واژه‌نامه کاربردی

اصطلاح	معنی در این جلسه
Pandas	کتابخانه پردازش داده جدولی در Python
DataFrame	جدول دوبعدی دارای ستون و Index
Series	ساختار یک‌بعدی با برچسب
Colab	محیط Notebook آنلاین گوگل
CSV	فرمت متنی مقادیر جداشده با جداکننده
Index	برچسب ردیف‌ها
`head` / `tail`	دیدن ابتدا و انتهای داده
`sample`	نمونه‌گیری از ردیف‌ها
`info`	اطلاعات نوع و مقدارهای غیرخالی
`describe`	آمار توصیفی
`loc`	انتخاب مبتنی بر برچسب
`iloc`	انتخاب مبتنی بر موقعیت
Boolean Mask	ماسک درست/نادرست برای فیلتر
`isna`	تشخیص مقادیر گمشده
`fillna`	جایگزینی مقدارهای گمشده
`dropna`	حذف براساس داده‌های گمشده
`drop_duplicates`	حذف تکراری‌ها طبق معیار
`merge`	اتصال داده بر اساس کلید
`concat`	چسباندن DataFrameها روی محور ردیف/ستون
`groupby`	گروه‌بندی برای تجمیع
`agg`	محاسبه یک یا چند تابع تجمیعی
`query`	فیلتر با عبارت متنی
`sort_values`	مرتب‌سازی براساس مقادیر
Pivot Table	جدول خلاصه چندبعدی
Fan-out	تکثیر ناخواسته ردیف پس از اتصال

۱۶. سوال‌های مرور

۱. Pandas چه کمکی به تحلیل داده جدولی می‌کند و چه محدودیتی در مقایسه با DBMS دارد؟

۱. تفاوت `Series` و `DataFrame` چیست؟

۲. `df.shape` چه خروجی‌ای می‌دهد؟

۳. `info()` و `describe()` چه تفاوتی دارند؟

۴. چرا ممکن است نتیجه `loc[1:3]` با `iloc[1:3]` متفاوت باشد؟

۵. برای انتخاب ستون‌های `Name` و `Age` چه می‌نویسیم؟

۶. چرا فیلتر چندشرطی Pandas به `&` و پرانتز نیاز دارد؟

۷. چگونه زنان کلاس اول را انتخاب می‌کنیم؟

۸. تفاوت ساخت ستون ثابت با ستون محاسباتی چیست؟

۹. تفاوت `concat(axis=0)` و `merge(on='Id')` چیست؟

۱۰. چه زمانی `merge` باعث Fan-out می‌شود؟

۱۱. `count()` در تجمیع چه تفاوتی با `size()` دارد؟

۱۲. چرا `dropna()` بدون تعیین ستون می‌تواند خطرناک باشد؟

۱۳. برای حذف رکوردهای تکراری چرا انتخاب کلید مهم است؟

۱۴. اگر بخواهیم میانگین سن به تفکیک جنسیت را محاسبه کنیم، دستور اصلی چیست؟

۱۷. پاسخ کوتاه سوال‌ها

۱. پردازش و تحلیل جدول در Python؛ با محدودیت حافظه و ویژگی‌های متفاوت از DBMS.

۲. Series یک‌بعدی و DataFrame دوبعدی است.

۳. یک Tuple شامل تعداد ردیف و ستون.

۴. `info` ساختار و مقادیر غیرخالی؛ `describe` آمار توصیفی.

۵. `loc` از برچسب و `iloc` از موقعیت استفاده می‌کند؛ انتهای برش نیز متفاوت است.

۶. دستور مناسب `df[['Name', 'Age']]` است.

۷. شرط‌های Series باید عنصربه‌عنصر با `&` یا `|` ترکیب شوند.

۸. از دستور `df.loc[(df['Sex']=='female') & (df['Pclass']==1)]` استفاده می‌کنیم.

۹. ستون ثابت یک مقدار را پخش می‌کند؛ ستون محاسباتی از داده ردیف‌ها نتیجه می‌گیرد.

۱۰. `concat` روی محور انتخابی می‌چسباند؛ `merge` براساس کلید رکوردها را مرتبط می‌کند.

۱۱. زمانی که کلید اتصال چندبار در یکی یا هر دو طرف تکرار شود.

۱۲. `count` مقدارهای غیرخالی؛ `size` تعداد ردیف‌های گروه.

۱۳. ممکن است ردیف‌های زیادی صرفاً به علت خالی‌بودن یک فیلد حذف شوند.

۱۴. تکراری‌بودن نام یا سایر ویژگی‌ها لزوماً به معنی رکورد تکراری کسب‌وکاری نیست.

۱۵. دستور محاسبه `df.groupby('Sex')['Age'].mean()` است.

۱۸. جمع‌بندی و مسیر تمرین

این جلسه نشان داد که مفاهیم آشنای SQL در Pandas با روش اجرایی متفاوتی بیان می‌شوند: انتخاب و فیلتر، محاسبه ستون، الحاق و اتصال، گروه‌بندی و پاک‌سازی. برای تمرین، ابتدا با یک DataFrame کوچک کار کنید؛ سپس روی مجموعه‌داده واقعی، شمارش ردیف‌ها و مقدارهای گمشده را پیش و پس از هر تبدیل کنترل کنید. این مرحله برای جلوگیری از خطاهای خاموش در پایپ‌لاین‌های داده اهمیت دارد.